

# EduMind: An AI-Powered Adaptive Learning Platform

## with Persistent Learner Modeling

Final Project Report for UCS503P

Mihir Khanna • Parth Khurana • Ojesh Srivastav  
Thapar Institute of Engineering and Technology

18 September 2026

### Abstract

This report documents the system delivered against the EduMind project proposal of 21 August 2026. The proposal argued that the deficiency in current AI study tools is not capability but *continuity*: a student moves between a syllabus, a chat assistant, a practice paper and a set of notes, and no tool retains a reliable picture of what they know, what they get wrong, or what they need next. EduMind addresses this by maintaining a persistent, concept-level model of the learner and using that model to condition every subsequent lesson, exploration and revision.

The delivered system implements the proposed architecture across all four layers. A LangGraph state machine routes each turn to one of nine specialised nodes; a fourteen-table PostgreSQL schema holds the concept graph, per-concept mastery, a misconception ledger and versioned learner snapshots; lessons are generated with a structure chosen per concept and learner, grounded in uploaded material by retrieval and streamed incrementally; side-questions fork the orchestrator checkpoint into independently resumable branches; a visual-need agent decides per concept whether a diagram, an interactive simulation, both or neither is warranted; and a deterministic spaced-repetition scheduler and objective grader operate with no model calls at all. Every model invocation passes through a single routing function that maps a closed task enumeration to one of three capability tiers. A suite of 48 asynchronous integration tests executing against a live PostgreSQL instance passes in full, and the complete stack was verified running on the date of this report.

The proposal's primary objective is met, as are three of its five numbered sub-objectives, including the digital learner twin with versioned snapshots. The shortfalls are concentrated in measurement rather than function: the per-call instrumentation described in §6.4 of the proposal was not implemented, so the primary evaluation metric — median model cost per active learning session — cannot be computed from stored records; the continuous-integration pipeline was not established; and the participant pilot was not conducted. Section 5 reports these plainly, and Section 9 sets out the bounded work that would close them.

---

## 1 Introduction

### 1.1 Motivation: continuity, not capability

A student working through a difficult topic today assembles their own toolchain. The syllabus tells them what to study; a video or a chat assistant explains it; a practice paper tests whether they understood; a fresh chat window handles the question they got stuck on; an image search supplies the picture the text omitted; and their notes, eventually, tell them what to revise. Each tool resolves one moment competently. None of them retains what matters across moments:

- **what the student knows** — how mastery of a concept has moved over time;
- **what the student gets wrong** — the misconception beneath a mistake, not merely the fact of it;
- **what the student needs next** — the unmet prerequisite, and the concept whose retention is decaying.

The consequence is that the learner rebuilds context from scratch at the start of every session, and the tools cannot improve with repetition because they retain nothing to improve upon. EduMind treats learning as a system with memory rather than a sequence of independent conversations. The design principle throughout is that the durable artefact is the learner model, not the transcript.

## 1.2 The four failures addressed

Section 3 of the proposal identified four recurring failures in the use of general-purpose conversational assistants as tutors. Table 1 records the mechanism delivered against each.

| Failure                      | Mechanism delivered                                                                                                                                                                                                            | Where                                                                    |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| Statelessness                | Per-concept mastery, confidence, retention score and misconception records persist in PostgreSQL and condition each subsequent lesson; versioned snapshots retain the learner’s trajectory across sessions.                    | <code>learner_concept_state</code> ,<br><code>twin_snapshots</code>      |
| Undiagnosed error            | Objective items grade deterministically; free-text answers are graded by a mid-tier model prompted to name the <i>reasoning error</i> , and a misconception node runs after every teaching and assessment turn.                | <code>misconception_node</code> ,<br><code>services/assessment.py</code> |
| Context loss on tangents     | A side-question forks the orchestrator checkpoint into a new thread identifier; the main lesson is preserved and both threads remain resumable and restorable from the database.                                               | <code>services/branching.py</code> ,<br><code>branches</code>            |
| Unbounded, unattributed cost | All calls pass a single routing function mapping tasks to capability tiers, with caching, batching and deterministic bypasses. Bounded by construction; <i>not instrumented</i> , so spend is not presently attributable (§5). | <code>app/models/router.py</code>                                        |

**Table 1:** The four identified failures and the delivered response.

## 1.3 Contribution

The contribution is one of integration and engineering, not of model research. No training or fine-tuning was attempted; hosted models are consumed as commodity endpoints behind a stable internal interface. What the project contributes is the assembly — a concept knowledge graph, an adaptive teaching engine, branching exploration, a visual learning layer, retrieval-grounded material and a versioned digital learner twin operating as one coherent system, in which the output of each session becomes the input to the next.

## 2 Objectives: status against the proposal

The primary objective is met. Learner state persists at concept granularity across sessions and demonstrably conditions what happens next: lesson depth is calibrated to recorded confidence, the revision queue is computed from stored retention scores, and each session closes by writing

| # | Objective (abridged from proposal §5)                                                                                                                                                 | Status               |
|---|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|
| — | <b>Primary:</b> deployable AI learning platform maintaining persistent concept-level learner state across sessions, and using that state to adapt teaching, exploration and revision. | <b>Delivered</b>     |
| 1 | Central AI orchestration and routing layer through which model-driven tasks are selected and managed — by Week 5.                                                                     | <b>Delivered</b>     |
| 2 | Measurable and bounded model cost per active learning session through model routing, caching and deterministic execution — by Week 12.                                                | <b>Partial</b>       |
| 3 | Persistent digital learner twin with versioned snapshots of mastery, confidence, misconceptions and learning history — by Week 10.                                                    | <b>Delivered</b>     |
| 4 | Measurable improvement in concept-level diagnostic performance between pre- and post-pilot assessment for at least 15 participants — by Week 13.                                      | <b>Not delivered</b> |
| 5 | CI pipeline building, testing and deploying to staging on every change — from Week 3 onward.                                                                                          | <b>Not delivered</b> |

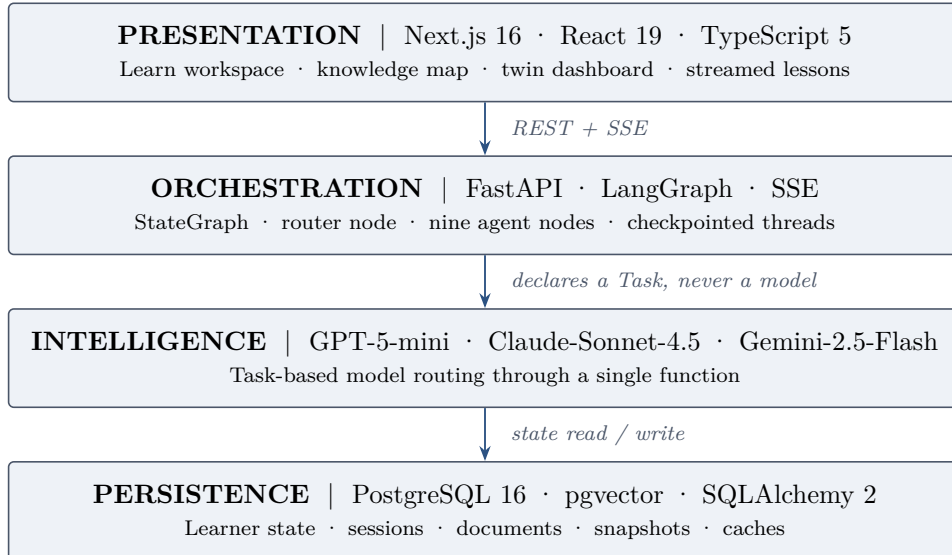
**Table 2:** Objective status. Objective 2 is marked partial because the routing, caching and deterministic-execution mechanisms it names are all in place, while the measurement that would evidence the bound is not.

a snapshot that the following session reads. The pattern in the shortfalls is consistent and worth naming directly — the objectives concerning *construction* were met and those concerning *measurement* were not.

### 3 The system as built

#### 3.1 Architecture

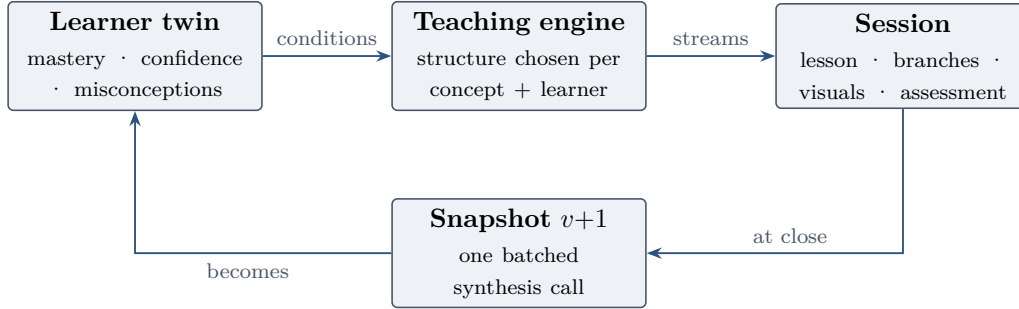
The system is organised as four layers, each with one responsibility.



**Figure 1:** The four-layer architecture as delivered. State lives in the database, not in a context window.

### 3.2 The memory loop

The architecturally significant property is not any single layer but the loop closed between them. A session consumes the learner model and, at its close, produces a revised one.



**Figure 2:** The memory loop. The next session begins from what the previous one established; the twin changes how the following lesson is taught.

### 3.3 The orchestration kernel

Every conversational turn enters one state machine, declared in `app/graph/graph.py`. The graph holds nine nodes — `router`, `diagnostic`, `teaching`, `branch`, `image`, `simulation`, `misconception`, `revision` and `twin_updater`. The entry point is the router node, which classifies intent using the cheapest capable model; a single conditional edge keyed on the classified mode dispatches to exactly one handler. Teaching and diagnostic paths additionally traverse the misconception node before terminating, so that every turn capable of revealing a faulty inference is examined for one.

Several paths reach a terminal state without any model call at all: revision scheduling, objective grading and study-order planning are ordinary program logic. This is the non-research stance of proposal §6.1 applied concretely — where a decision can be taken by deterministic code, it is.

### 3.4 The digital learner twin

Objective 3 is delivered. The twin is a versioned profile, not a transcript. During a session the system accumulates answers, confidence signals, mistakes and the set of concepts visited. At session close a single batched synthesis call — never one per message — produces a new snapshot row containing a cognitive profile, a behavioural profile and the knowledge state at that instant. Snapshots are versioned, so the dashboard renders both the current profile and a diff against its predecessor, alongside the open misconception ledger.

A deliberate resilience property is implemented here: should the synthesis call fail, the knowledge snapshot is still written, with the behavioural profile marked unavailable for that session. A failed summary never costs the learner their recorded progress, and this behaviour is covered by an explicit test.

### 3.5 The adaptive teaching engine

Lesson *structure* is chosen per concept and learner rather than fixed by a template. The engine emits open-ended sections — intuition, worked example, common mistakes, practice, interactive question, summary, and others as the concept warrants — and calibrates depth to recorded confidence, scaffolding more slowly where confidence is low and compressing where it is high. Lessons stream to the client incrementally over server-sent events, so text appears as it is produced rather than after a long silence.

Long-form generations use a `<<section>>` marker format rather than JSON. This is a correctness decision taken from experience: unescaped quotation marks and newlines in free-form model prose break JSON parsing reliably, whereas a line-oriented marker format degrades gracefully. Short structured outputs such as quizzes still use JSON, where the failure mode does not arise.

### 3.6 Branching as a checkpoint fork

A side-question neither restarts a conversation nor derails the lesson in progress. Because the orchestrator keys state by thread identifier, allocating a new identifier and replaying against the same compiled graph *is* the fork; no manual state copying is involved. Branch content and history are additionally persisted to the `branches` table, so a branch is restored when its topic is reopened. The schema permits a branch with a null parent block but a concept association, which supports a free-form thread opened under a topic rather than from one specific lesson block.

### 3.7 The visual and AI workspace

A dedicated visual-need agent decides, once per concept and cached thereafter, whether a static diagram, an interactive simulation, both or neither would genuinely help. The judgement is made independently for the two media, so both may fire or neither. Diagrams are retrieved through an external image search, segmented once by the vision tier and inserted inline; a region of a diagram may be cropped and carried into its own branch, which is the mechanism by which a visual question becomes a resumable exploration. Simulations are generated as sandboxed code and executed in an isolated frame.

The knowledge map is a custom infinite canvas — pan, zoom, bezier edges, minimap, focus mode — rendering the concept graph with each node coloured by mastery and ringed by confidence. It was implemented directly rather than through a graph library, which is why the frontend carries no such dependency.

### 3.8 Retrieval grounding

Uploaded PDFs and slide decks are chunked into `document_chunks` and retrieved at lesson-generation time, defaulting to the concept name as the query, so a lesson is grounded in the student’s own material where such material exists. Retrieval currently runs on PostgreSQL full-text search rather than vector similarity, because the hosted inference provider exposes no embeddings endpoint. The `pgvector` column is present and unpopulated, serving exactly the seam described in §8 of the proposal: semantic retrieval can replace keyword retrieval without migrating application logic.

### 3.9 Deterministic subsystems

**Spaced repetition.** `services/revision.py` implements a simplified SM-2 variant with no model calls. The stored `retention_score` plays the easiness-factor role, defaulting to 2.5 with a floor of 1.3. A review quality  $q$  is mapped from a 0..1 score onto SM-2’s 0..5 scale and easiness updated as

$$E' = \max(1.3, E + (0.1 - (5 - q)(0.08 + 0.02(5 - q)))) .$$

The interval is derived from the mastery band rather than a repetition count — one day below 0.3 mastery, three days below 0.6, and  $\max(6, \lceil 6E \rceil)$  days above — so shaky concepts resurface quickly and secure ones back off. Mastery itself updates by an exponentially weighted moving average with weight 0.3.

**Objective grading** executes in process; only free-text answers reach a model. **Study-order planning** over the concept graph is ordinary traversal logic.

### 3.10 Model routing and cost discipline

Feature code declares a task from a closed enumeration and never names a model; `app/models/router.py` alone resolves a task to a hosted bot. This was verified by inspection: no feature module imports the inference SDK.

| Task                             | Tier / model              | Invocation pattern                                          |
|----------------------------------|---------------------------|-------------------------------------------------------------|
| <code>classify_intent</code>     | cheap — GPT-5-mini        | Every conversational turn; the hot path.                    |
| <code>generate_quiz</code>       | cheap — GPT-5-mini        | Once per assessment.                                        |
| <code>classify_visual</code>     | cheap — GPT-5-mini        | Once per concept, then cached.                              |
| <code>track_misconception</code> | cheap — GPT-5-mini        | After each teaching or assessment turn.                     |
| <code>grade_answer</code>        | mid — Claude-Sonnet-4.5   | Free-text answers only; objective items bypass the model.   |
| <code>teach_concept</code>       | mid — Claude-Sonnet-4.5   | Lessons, branch replies, simulation code generation.        |
| <code>onboard_dynamic</code>     | mid — Claude-Sonnet-4.5   | Unknown subjects only; result persisted and reused.         |
| <code>twin_summarize</code>      | mid — Claude-Sonnet-4.5   | Once per session close, batched.                            |
| <code>diagram_segment</code>     | vision — Gemini-2.5-Flash | At most once per image; crops hashed and cached separately. |

**Table 3:** The delivered routing table, using the models named in §6.2 of the proposal. Every high-frequency task is confined to the cheap tier and the vision tier is invoked at most once per image.

Four disciplines bound spend, as set out in §6.3 of the proposal, and all four are implemented:

- 1. Cheap on the hot path.** Intent classification, which fires on every turn, uses the cheapest capable tier, as do misconception tracking, visual classification and quiz generation.
- 2. Deterministic where possible.** Objective grading, revision scheduling and planning incur no model cost whatsoever.
- 3. Cache before compute.** Lessons are cached under a key derived from concept identity and a mastery band, `teach:{concept_id}:{style}:{band}`, where the band partitions confidence at 0.34 and 0.67, so a lesson is regenerated only for a materially different learner. Simulations are cached per concept and cohort, image segmentations once per image, and arbitrary crops under a content hash. Where a lesson is grounded in a student’s own uploaded material the shared cache is deliberately bypassed, since that generation is no longer cohort-equivalent.
- 4. Batch expensive work.** Twin synthesis is one call per session close, not one per message.

**What is absent.** The router emits no telemetry. There is no record of task, tier, token counts, latency, cache-hit flag or session identifier for any call, and no table in which such a record could be stored. The consequences are examined in §5.

## 4 Implementation and verification

### 4.1 Data model

Fourteen tables under Alembic control across three migrations.

| Tables                          | Role                                                                                                                                  |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| users, learner_profiles         | Accounts and onboarding preferences.                                                                                                  |
| concepts, learner_concept_state | Prerequisite-linked concept graph; per-learner mastery, confidence, retention score and misconception ledger.                         |
| sessions                        | Session boundaries, which drive snapshot batching.                                                                                    |
| lesson_blocks                   | Generated lesson content, keyed per concept and cohort.                                                                               |
| branches                        | Forked threads; nullable <code>parent_block_id</code> with a <code>concept_id</code> column permits a free-form thread under a topic. |
| documents, document_chunks      | Uploaded material chunked for retrieval; <code>embedding</code> reserved for a future provider.                                       |
| diagrams, crop_cache            | Images, segmented regions, and a hashed-crop question cache.                                                                          |
| simulation_cache                | Generated simulation code, cached per concept and cohort.                                                                             |
| twin_snapshots                  | Versioned learner profiles supporting the diff view.                                                                                  |
| subject_templates               | Built-in and dynamically generated curricula.                                                                                         |

## 4.2 Automated tests

The suite comprises 48 asynchronous integration tests across eleven modules, totalling 1,171 lines. They execute against a live PostgreSQL instance rather than a mocked database, so schema, migration and query behaviour are exercised as deployed. Coverage spans onboarding, assessment, teaching, branching, graph dispatch, images, simulations, planning, revision, routing and the twin — including the failure path in which synthesis fails but the knowledge snapshot must still be written.

| Measure             | Value                                        |
|---------------------|----------------------------------------------|
| Tests executed      | 48                                           |
| Tests passed        | 48                                           |
| Wall-clock duration | 4.53 s                                       |
| Database            | live PostgreSQL 16 with pgvector, not mocked |
| Date of run         | 18 September 2026                            |

## 4.3 Runtime verification

The full stack was brought up and confirmed serving on the date of this report: the containerised database on port 5433, the backend on port 8000 with interactive API documentation responding, and the frontend development server on port 3000 responding.

The backend is started through `backend/run.py` rather than the server CLI. On Windows the asynchronous PostgreSQL driver fails under the default proactor event loop; the entry point installs a selector event-loop policy before the server acquires the loop, and the test configuration applies the same policy so that tests match deployed behaviour.

## 4.4 Scale of the implementation

| Component            | Files     | Lines        |
|----------------------|-----------|--------------|
| Backend application  | 40        | 3,673        |
| Backend tests        | 11        | 1,171        |
| Frontend application | 18        | 3,980        |
| <b>Total</b>         | <b>69</b> | <b>8,824</b> |

## 5 Evaluation against the proposal’s criteria

### 5.1 Primary metric: median model cost per active session

**Status: not computable from stored records.**

The proposal specified this metric as primary because it is the falsifiable claim on which continuous operation rests, and specified its attribution method exactly: each routed call writes a record of task, tier, token counts, latency, cache-hit flag and session identifier, and cost is obtained by joining those records to published token prices. Section 6.4 of the proposal further committed that instrumentation would be designed before implementation rather than retrofitted.

No such records are written. Inspection of the routing module confirms it emits no telemetry, and no table exists to receive it. The consequence should be stated without softening: the US\$0.05 target is neither met nor refuted, and no figure for it may honestly be reported. The proposal anticipated missing the target and provided for reporting a measured figure as a negative result; it did not anticipate being unable to take the measurement, which is the weaker position the project occupies.

Two observations remain defensible on the evidence, and they concern the mechanism rather than the outcome. First, the routing table is a genuine single point of control, verified by inspection — every call is in principle attributable the moment logging exists, and no feature module would need to change for it to be. Second, the tier assignment in Table 3 confines every high-frequency task to the cheap tier and restricts the vision tier to at most one call per image, while three of the four disciplines in §3.10 remove work from models altogether. These are the design decisions on which a low per-session cost would depend, and they are in place. What is missing is the evidence, not the mechanism.

### 5.2 Secondary metrics

| Metric                                                        | Status               | Position                                                                                                                                                                                                                                                                                                                                                                                             |
|---------------------------------------------------------------|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Cache-service rate<br>(target $\geq 40\%$ )                   | <b>Partial</b>       | Four caching layers are implemented — cohort-keyed lessons, simulations per concept and cohort, per-image segmentation, and hashed crop queries — of which three are database-backed and durable. No hit or miss counter exists, so the rate is unmeasured. The lesson response cache is additionally in-process, so it does not survive a restart and would not be shared across backend instances. |
| Learning gain                                                 | <b>Not delivered</b> | Requires the pilot. The underlying data — per-concept mastery before and after — is stored and would support the paired comparison once collected under controlled conditions.                                                                                                                                                                                                                       |
| Responsiveness<br>(95th percentile to first token $\leq 3$ s) | <b>Not delivered</b> | Lessons stream over server-sent events as specified and streaming was confirmed functional, but no latency timings are recorded.                                                                                                                                                                                                                                                                     |
| Reliability ( $\geq 99\%$ in pilot hours)                     | <b>Not delivered</b> | No declared pilot window and no uptime monitoring.                                                                                                                                                                                                                                                                                                                                                   |

### 5.3 Pilot validation

The pilot described in §7.3 of the proposal — 15 to 25 volunteers across two subjects, with baseline and equivalent-form post-diagnostics either side of a two-week window — was not conducted.

The system was nonetheless exercised well beyond the automated suite. Excluding accounts created by test fixtures, the database holds 11 real accounts associated with 196 sessions and

10 versioned learner snapshots. This is reported for what it is: development and demonstration usage. The participants were not recruited, no baseline instrument was administered, and no learning-gain inference may be drawn from it. Its evidentiary value is confined to showing that the end-to-end flows function under repeated real use rather than only under test conditions.

## 5.4 Functional evidence

Where quantitative evaluation is unavailable, the functional position is stated instead. Every stage of the core workflow in §4.2 of the proposal — subject selection and graph construction, diagnostic with mastery write-back, state-conditioned streamed lessons, branch creation and return, and session-close snapshot with revision scheduling — is implemented, covered by integration tests, and was exercised during the usage described above.

## 6 Deviations and shortfalls

| Area                            | Proposed                                                                                               | Delivered, and why                                                                                                                                                                                                                                                                 |
|---------------------------------|--------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Call instrumentation (§6.4)     | Per-call record of task, tier, tokens, latency, cache-hit and session; designed before implementation. | Absent. The primary metric is consequently uncomputable. No technical obstacle was identified; the work was not undertaken. This is the principal shortfall.                                                                                                                       |
| CI/CD (§6.5, obj. 5)            | Build, test, lint and staging deployment on every change, from Week 3.                                 | Absent; no pipeline definition exists in the repository. The test suite passes locally and deployment configuration for the target platforms is present, but neither is automated, and the repository history does not evidence the weekly iteration cadence the proposal assumed. |
| Pilot (§7.3, obj. 4)            | 15–25 volunteers, pre- and post-diagnostics over a two-week window.                                    | Not conducted. Development and demonstration usage is reported instead, and explicitly not as a study.                                                                                                                                                                             |
| Orchestrator checkpointing (§9) | Graph orchestration with a database-backed checkpointer.                                               | In-process checkpointer. Branch content and history persist independently in PostgreSQL and are restored on reopen, so the learner’s durable record is unaffected; what does not survive a restart is in-flight graph state.                                                       |

| Area                   | Proposed                                                                                   | Delivered, and why                                                                                                                                                                                                                 |
|------------------------|--------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Response cache         | Cohort-keyed caching of expensive generations.                                             | Implemented and correctly keyed, but the lesson cache is in-process. Its interface is deliberately shaped for a shared backing store, so substitution is contained to one module. Simulation, diagram and crop caches are durable. |
| Retrieval              | Reserved vector column; keyword retrieval initially.                                       | As proposed. Full-text retrieval is in use and the vector column awaits an embeddings provider, which the hosted inference API does not expose. This is the anticipated path, not a shortfall.                                     |
| Workspace tools (§4.1) | Explanations, summaries, quizzes, flashcards, mind maps and context-aware visual learning. | Explanations, summaries, quizzes, the knowledge map and the context-aware visual layer are delivered. <b>Flashcards were not implemented.</b>                                                                                      |

## 7 Risks: which materialised

**Model availability drift (materialised; mitigation succeeded).** The proposal confined model names to the routing table alone and provided a probe script, so that a withdrawn or access-restricted model could be substituted without touching feature code. This risk did occur during development: two models originally intended for the mid and vision tiers refused programmatic access. Substitution of nearest-capable equivalents was a two-line change to the routing table, and `probe_bots.py` re-verifies availability on demand. The models named in §6.2 of the proposal are precisely those running in the delivered system. This is direct evidence that the architectural boundary the project set out to build delivers the property claimed for it.

**Cost target not met (materialised in a worse form).** The proposal provided for reporting a measured figure as a negative result should the target be missed. The realised situation is weaker: the measurement apparatus was not built, so no figure exists to report either way. The mitigation presupposed instrumentation and therefore did not apply.

**Insufficient pilot recruitment (materialised; mitigation did not apply).** The proposal reasoned that cost, cache and latency metrics do not depend on cohort size and would remain valid under under-recruitment. As none of those three is instrumented, the fallback yields nothing.

**Evaluation ambiguity (definitions fixed, never exercised).** Session boundaries are defined and recorded in the `sessions` table, and mastery transitions are deterministic. The definitions exist, but with no metric pipeline consuming them they were never put to use.

**Free-tier operational limits (not reached).** No pilot was run against hosted staging.

**Scope growth (did not materialise).** The visual learning layer, knowledge-map exploration and simulation generation all fall within §4.1 and §10.2 of the proposal, and the exclusions in §10.3 — mobile-native clients, LMS integration, multi-tenant administration, offline operation and any model training — were observed without exception.

## 8 Scalability assessment

The proposal’s scalability claims are assessed against the delivered code rather than restated.

- **Horizontal API scaling.** Partially supported. The HTTP surface is stateless and instances could be added, but two components hold process-local state — the orchestrator checkpointer and the lesson response cache — so a multi-instance deployment would today show inconsistent cache behaviour and non-portable graph state. Both have a defined substitution path and neither requires a schema change.
- **Indexed queries and pagination.** Concept, lesson and branch access is by indexed foreign key; the concept table already holds 2,886 rows without degradation in the test suite.
- **Cohort-keyed caching.** Structurally sound. The key is concept identity crossed with a three-band mastery partition and a style preference, so the marginal generation cost of an additional learner in an occupied band is zero, and the benefit grows with cohort size as claimed. Absent counters, the realised rate is unknown.
- **Vector seam.** Present and unused, as intended.
- **Long-lived process requirement.** Confirmed and reinforced: the backend must run as a persistent process, since it holds orchestrator checkpoints and streams responses for longer than typical serverless limits permit. The deployment configuration reflects this.

## 9 Remaining work

Ordered by dependency and by contribution to the proposal’s outstanding claims.

1. **Instrument the router.** Add a call-record table capturing task, tier, prompt and completion token counts, latency, cache-hit flag, session and learner identifiers, written from the two functions through which all calls already pass. This is bounded work precisely because the choke point exists: no feature module requires modification. It is the precondition for objective 2 and for two of the four secondary metrics.
2. **Compute the primary metric.** Join call records to published token prices and report the median per active session, an active session being one containing at least one completed lesson or assessment.
3. **Count cache outcomes** at each of the four caching layers and report the service rate against the 40% target.
4. **Record first-token latency** on the streaming path and report the 95th percentile.
5. **Establish the CI pipeline.** Build, run the existing 48 tests, lint, and deploy to staging on every change; the suite already passes and needs only a database service in the runner.
6. **Move the checkpointer and lesson cache to shared backing stores**, closing the horizontal-scaling gap in §8.
7. **Implement flashcards** to complete the workspace tools listed in §4.1 of the proposal.
8. **Conduct the pilot** once items 1–5 are in place, since the instrumentation is what makes the pilot worth running.

## 10 Conclusion

EduMind was proposed as an integration contribution: a concept knowledge graph, an adaptive teaching engine, branching exploration, a visual learning layer, retrieval-grounded material and a

versioned digital learner twin operating as one system in which learner state persists and shapes what happens next. That system exists and works. Mastery, confidence and misconceptions are recorded per concept and survive across sessions; lesson structure and depth are chosen against that record; a side-question forks into a resumable branch without disturbing the lesson; a visual-need agent decides independently whether a diagram or a simulation earns its place; revision is scheduled by deterministic logic over stored retention; and each session closes by writing a snapshot that the next session reads. Forty-eight integration tests pass against a live database, and the stack runs.

The cost-governance mechanism is likewise built as an architectural boundary rather than a convention, and was validated under adversarial conditions when two intended models became unavailable and were replaced by a two-line change with no effect on feature code. What was not built is the instrumentation that would have turned that boundary into evidence. A system whose per-session cost cannot be measured cannot be shown to be bounded, however carefully its routing is designed, and the project therefore cannot report its primary metric. The architecture is arranged so that this evidence is inexpensive to obtain — the calls are already funnelled through one place — but it was not obtained within the project window. That, together with the absent CI pipeline and the uncondacted pilot, is the honest limit of what this work demonstrates.

## A API surface

| Router      | Endpoints                                                                                                |
|-------------|----------------------------------------------------------------------------------------------------------|
| auth        | POST /signup, POST /login                                                                                |
| onboarding  | GET /subjects, GET /questions, POST /complete                                                            |
| graph       | GET /plan, POST /concepts, GET /graph                                                                    |
| teach       | GET /lessons, GET /stream (SSE)                                                                          |
| assessment  | POST /start, POST /answer                                                                                |
| branches    | POST /, GET /, GET /concept/{id}, GET /{id}/history, POST /{id}/message, DELETE /{id}                    |
| images      | POST /upload, POST /fetch, POST /generate, GET /concept/{id}, GET /{id}, POST /{id}/ask, POST /{id}/crop |
| simulations | GET /cached, POST /                                                                                      |
| revision    | GET /due, POST /review                                                                                   |
| twin        | POST /{user_id}/snapshot, GET /{user_id}, GET /{user_id}/history, GET /{user_id}/diff                    |
| sessions    | GET /, POST /start, POST /{id}/end                                                                       |
| profile     | GET /{user_id}, PUT /{user_id}                                                                           |
| documents   | POST /roadmap, POST /upload, GET /search                                                                 |

## B Reproduction

1. **Database.** `docker compose up -d postgres` — PostgreSQL 16 with pgvector, published on port 5433.
2. **Backend.** From `backend/`: create a virtual environment, install `requirements.txt`, copy `.env.example` to `.env` and set the inference API key, apply migrations with `alembic upgrade head`, then start with `python run.py`. The entry point must be used in preference to the server CLI on Windows, for the event-loop reason given in §4.3.
3. **Frontend.** From `frontend/`: `npm install`, then `npm run dev`.
4. **Tests.** From `backend/`: `python -m pytest -q`.

The backend serves on port 8000 with interactive documentation at `/docs`; the frontend serves on port 3000.